ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO BÀI TẬP LỚN

Môn học: Kiến trúc máy tính tiên tiến

Thiết kế máy tính RISC-V 32-bit đơn chu kỳ và pipeline 5 tầng trên Logisim-evolution

Nhóm sinh viên thực hiện

STT	Họ và tên	MSHV
1	Quyền Đức Công	20242423M
2	Bùi Phạm Sơn Hà	20251053M
3	Nguyễn Đức Anh	20252594M

Giảng viên hướng dẫn: TS. Hoàng Văn Hiệp

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 11/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 1

1.1 Mục tiêu và yêu cầu 1

1.2 Đối chiếu yêu cầu với sản phẩm 1

CHƯƠNG 2. TẬP LỆNH RV32I VÀ KIẾN TRÚC TỔNG THỂ 3

2.1 Trạng thái kiến trúc và định dạng lệnh 3

2.2 Sơ đồ khối và các quyết định thiết kế 3

2.3 Bản đồ bộ nhớ 4

CHƯƠNG 3. THIẾT KẾ CÁC MODULE 5

3.1 ALU và ALUControl 5

3.2 RegisterFile 6

3.3 ImmGen 6

3.4 Control 6

3.5 BranchUnit 7

3.6 LoadUnit, StoreUnit và AddrDecode 7

3.7 IOController 8

3.8 PerfCounters 8

CHƯƠNG 4. CPU ĐƠN CHU KỲ 9

4.1 Datapath 9

4.2 Ví dụ: lệnh `lw t4, 8(sp)` 10

4.3 Đường găng và giới hạn của thiết kế đơn chu kỳ 10

CHƯƠNG 5. CPU PIPELINE 5 TẦNG VÀ XỬ LÝ HAZARD 11

5.1 Chia datapath thành năm tầng 11

5.2 Thanh ghi pipeline 11

5.3 Ba loại hazard 12
5.3.1 Forwarding và bypass WB→ID 12
5.3.2 Load-use: stall một chu kỳ 13
5.3.3 Control hazard: dự đoán không nhảy và flush 13
5.4 Bản NoHazard và cách chèn NOP 13
CHƯƠNG 6. KIỂM THỬ 15
6.1 Toolchain 15
6.2 Test vector từng module 15
6.3 Chương trình test toàn CPU 16
6.4 Kết quả 16
CHƯƠNG 7. KẾT QUẢ VÀ ĐÁNH GIÁ 18
7.1 Số chu kỳ của ba phiên bản 18
7.2 Thời gian thực thi theo giả định trễ 19
7.3 Một lần chạy cụ thể 19
CHƯƠNG 8. PHẦN MỞ RỘNG, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN 20
8.1 Phần mở rộng đã làm 20
8.2 Hạn chế 20
8.3 Hướng phát triển 20
TÀI LIỆU THAM KHẢO 21
PHỤ LỤC 23
A. HƯỚNG DẪN SỬ DỤNG PROJECT 23
A.1 Mở project và chạy mô phỏng 23
A.2 Nạp chương trình mới 23
A.3 Chạy test 23
A.4 Dựng lại báo cáo 24

A.5 Kịch bản demo 24

DANH MỤC HÌNH VẼ

Hình 3.1 Circuit `ALU`: mười phép toán tính song song, MUX chọn theo funct3 5

Hình 3.2 Circuit `StoreUnit`: ghép byte hoặc halfword mới vào word cũ 8

Hình 4.1 Mạch top-level của CPU đơn chu kỳ trong Logisim 9

Hình 5.1 Mạch top-level của CPU pipeline. Pipeline Monitor ở góc dưới hiển thị PC và lệnh ở từng tầng cùng các tín hiệu `Stall`, `LoadUse`, `E_Redirect`, `FwdA`, `FwdB` 11

Hình 7.1 Tăng tốc so với SC theo giả định trễ; đường đứt nét là mức lý tưởng khi CPI = 1 19

DANH MỤC BẢNG BIỂU

Bảng 1.1 Yêu cầu của đề và phần hiện thực tương ứng 2

Bảng 2.1 Sáu định dạng lệnh RV32I 3
Bảng 2.2 Các quyết định thiết kế 4
Bảng 2.3 Bản đồ bộ nhớ 4

Bảng 3.1 Mã hóa `ALUCtrl` 5
Bảng 3.2 Tín hiệu điều khiển theo nhóm lệnh 7

Bảng 5.1 Các trường của thanh ghi pipeline 12
Bảng 5.2 Giản đồ pipeline của 7 lệnh trong `t06_hazard.s` 14

Bảng 6.1 Kết quả test vector (đạt / tổng số); ba khối cuối chỉ có ở PL . . 16
Bảng 6.2 Kết quả chạy `run_tests.py` 16

Bảng 7.1 Số lệnh và số chu kỳ trên SC, NH và PL 18

Bảng 8.1 Hướng phát triển và cách đo lợi ích 20

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ALU	Khối số học và logic (Arithmetic Logic Unit)
CPI	Số chu kỳ trung bình cho một lệnh (Cycles Per Instruction)
CPU	Bộ xử lý trung tâm (Central Processing Unit)
GPIO	Cổng vào ra đa dụng (General Purpose Input/Output)
IF, ID, EX, MEM, WB	Năm tầng pipeline: nạp lệnh, giải mã, thực thi, truy cập bộ nhớ, ghi lại
ISA	Kiến trúc tập lệnh (Instruction Set Architecture)
ISS	Trình mô phỏng ở mức lệnh (Instruction Set Simulator), ở đây là golden model `rvsim.py`
NH	Bản pipeline không có forwarding và load-use stall, file `RV32I_Pipeline_NoHazard.circ`
NOP	Lệnh không làm gì; trong RV32I là `addi x0, x0, 0`
PC	Thanh ghi địa chỉ lệnh (Program Counter)
PL	Bản pipeline 5 tầng đầy đủ (`RV32I_Pipeline.circ`)
RAM	Bộ nhớ đọc ghi; ở đây là bộ nhớ dữ liệu `DMEM`
RAW	Phụ thuộc đọc sau ghi (Read After Write), dạng data hazard trong pipeline
ROM	Bộ nhớ chỉ đọc; ở đây là bộ nhớ lệnh `IMEM`
RV32I	Tập lệnh số nguyên cơ sở 32 bit của RISC-V

Thuật ngữ	**Ý nghĩa**
SC	Bản CPU đơn chu kỳ (`RV32I_SingleCycle.circ`)
UART	Giao tiếp nối tiếp không đồng bộ (Universal Asynchronous Receiver-Transmitter)

CHƯƠNG 1. GIỚI THIỆU

1.1 Mục tiêu và yêu cầu

Đề bài yêu cầu thiết kế, mô phỏng và kiểm thử một máy tính 32 bit theo kiến trúc tập lệnh RISC-V RV32I [1], gồm CPU, bộ nhớ chương trình, bộ nhớ dữ liệu và các khối điều khiển. Phải có hai phiên bản: một CPU đơn chu kỳ và một CPU pipeline 5 tầng IF, ID, EX, MEM, WB xử lý đúng data hazard và control hazard cơ bản. Dữ liệu, lệnh, thanh ghi và PC đều rộng 32 bit, có 32 thanh ghi. Chương trình phải chạy từ lệnh nạp vào bộ nhớ lệnh, không được chỉnh tay giá trị thanh ghi. Công cụ là Logisim-evolution [2].

Nhóm làm ba file mạch, đều mở bằng Logisim-evolution 4.1.0:

- `circ/RV32I_SingleCycle.circ`: CPU đơn chu kỳ, trong báo cáo gọi tắt là SC.
- `circ/RV32I_Pipeline.circ`: CPU pipeline 5 tầng có forwarding, bypass WB→ID, stall khi load-use và flush khi rẽ nhánh, gọi tắt là PL. Đây là phiên bản cuối cùng theo yêu cầu đề.
- `circ/extra/RV32I_Pipeline_NoHazard.circ`: cùng pipeline nhưng tắt forwarding và load-use stall, gọi tắt là NH. Bản này chỉ chạy đúng khi assembler chèn lệnh `nop`; nhóm dùng nó để đo xem phần cứng xử lý hazard đem lại bao nhiêu.

1.2 Đối chiếu yêu cầu với sản phẩm

Bảng 1.1 liệt kê từng yêu cầu của đề và vị trí đáp ứng trong project.

Bảng 1.1: Yêu cầu của đề và phần hiện thực tương ứng

Yêu cầu	Hiện thực
Data, lệnh, thanh ghi, PC 32 bit; 32 thanh ghi	Datapath 32 bit, `RegisterFile` có x0 cố định bằng 0 và x1 đến x31
Lệnh số học, logic (10 lệnh R)	`ALU`, `ALUControl`; chương trình `t01_alu.s`
Lệnh immediate (9 lệnh I)	`ImmGen` và `ALU`; `t01_alu.s`
LW, SW; tùy chọn LB, LH, LBU, LHU, SB, SH	`LoadUnit`, `StoreUnit`, RAM `DMEM`; `t05_mem.s`. Làm cả phần tùy chọn
BEQ, BNE, JAL, JALR; tùy chọn BLT, BGE, BLTU, BGEU	`BranchUnit` và MUX chọn PC kế tiếp; `t03_branch.s`, `t04_jump.s`. Làm cả phần tùy chọn
Thêm ngoài danh sách tối thiểu	LUI, AUIPC; ECALL và EBREAK dùng làm lệnh dừng (halt)
Phiên bản đơn chu kỳ	`circ/RV32I_SingleCycle.circ` (chương 4)
Pipeline 5 tầng, xử lý hazard	`circ/RV32I_Pipeline.circ`: `ForwardUnit`, `IDBypass`, `HazardUnit` (chương 5)
Chạy từ lệnh nạp vào bộ nhớ lệnh	ROM `IMEM`; thanh ghi chỉ đổi khi lệnh thực thi
Công cụ assembly sang mã máy nạp ROM	`tools/rvasm.py`: sinh file hex hoặc nạp thẳng vào ROM trong file `.circ`
Nâng cao: GPIO/UART	`IOController`: UART TX ra TTY, UART RX từ Keyboard, GPIO ra LED 7 đoạn và 8 LED, GPIO vào 8 công tắc DIP
Nâng cao: tăng hiệu năng pipeline	Forwarding từ EX/MEM và MEM/WB, bypass WB→ID, tín hiệu `UseRs1`/`UseRs2` để không stall khi lệnh không đọc thanh ghi
Bộ test từng module và toàn CPU	13 file test vector trong `tests/vectors/`; 12 chương trình trong `programs/` và 25 chương trình ngẫu nhiên trong `programs/fuzz/`; `tools/run_tests.py` (chương 6)
README	`README.md`; phụ lục A

Các mục nâng cao chưa làm là dự đoán nhánh động, cache, nhân chia, ngắt và số thực dấu phẩy động. Chương 8 ghi lại phần mở rộng đã làm, các hạn chế và hướng làm tiếp.

Mã nguồn chia thành `circ/` (file mạch), `generator/` (Python sinh lại file mạch), `programs/` (chương trình test), `tests/vectors/` (test vector từng module), `tools/` (assembler, golden model, bộ chạy test) và `latex/` (báo cáo này). Cách mở project và chạy test ở phụ lục A.

CHƯƠNG 2. TẬP LỆNH RV32I VÀ KIẾN TRÚC TỔNG THỂ

2.1 Trạng thái kiến trúc và định dạng lệnh

RV32I có 32 thanh ghi x0 đến x31, mỗi thanh ghi 32 bit, trong đó x0 luôn đọc ra 0 và mọi lệnh ghi vào x0 bị bỏ qua [1]. Bộ nhớ đánh địa chỉ theo byte, kiểu little-endian. Mỗi lệnh dài 4 byte nên PC tăng 4 sau mỗi lệnh. ROM và RAM của Logisim lưu mỗi ô một word, nên chỉ số ô bằng địa chỉ byte bỏ 2 bit thấp: `IMEM` dùng `PC[11:2]`, `DMEM` dùng `Addr[13:2]`.

Mọi lệnh có opcode ở bit [6:0]. Các trường rd [11:7], funct3 [14:12], rs1 [19:15], rs2 [24:20] luôn ở cùng vị trí trong mọi định dạng có trường đó (bảng 2.1). Nhờ vậy register file đọc rs1, rs2 song song với việc giải mã. Bit dấu của immediate luôn là bit 31, nên phép mở rộng dấu bắt đầu được ngay mà không cần biết định dạng.

Bảng 2.1: Sáu định dạng lệnh RV32I

Định dạng	[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

Hai CPU hỗ trợ toàn bộ RV32I base trừ FENCE và các lệnh CSR: 10 lệnh R, 9 lệnh I số học, 5 lệnh load, 3 lệnh store, 6 lệnh rẽ nhánh có điều kiện, JAL, JALR, LUI và AUIPC. ECALL và EBREAK được quy ước là lệnh dừng CPU, để bộ test có điểm kết thúc rõ ràng.

2.2 Sơ đồ khối và các quyết định thiết kế

Datapath gồm thanh ghi `PC`, ROM `IMEM`, khối giải mã `Control` và `ImmGen`, `RegisterFile`, `ALU` cùng `BranchUnit`, RAM `DMEM` với `LoadUnit`, `StoreUnit`, và MUX chọn giá trị ghi lại. Các circuit con này dùng chung cho SC và PL; PL thêm 4 thanh ghi pipeline và 3 khối xử lý hazard. Bảng 2.2 ghi các quyết định thiết kế chính.

Bảng 2.2: Các quyết định thiết kế

Quyết định	Lý do và đánh đổi
Kiến trúc Harvard: `IMEM` là ROM, `DMEM` là RAM riêng	Nạp lệnh và load/store không tranh nhau một cổng bộ nhớ, tránh structural hazard giữa tầng IF và MEM. Đổi lại chương trình không tự sửa mã được.
ALU tính luôn địa chỉ đích	Với branch và JAL, đầu vào A của ALU là PC; với JALR là rs1. Đích nhảy = kết quả ALU AND `0xFFFFFFFE`. Không cần bộ cộng PC + imm riêng; phép so sánh rs1 với rs2 vì thế làm ở `BranchUnit`.
Lệnh `0x00000000` là NOP	Decoder trong `Control` chỉ bật khi `opcode[1:0]` = 11, nên lệnh toàn 0 cho mọi tín hiệu điều khiển bằng 0. Flush một tầng pipeline chỉ cần nạp 0.
SB, SH bằng đọc, sửa rồi ghi cả word	RAM của Logisim chỉ có byte enable khi đọc đồng bộ, không dùng được cho CPU đơn chu kỳ.
Memory-mapped I/O	UART và GPIO là vài địa chỉ đặc biệt; CPU dùng chính `lw`, `sw`, `lb`, `sb` để điều khiển, không cần lệnh mới.

2.3 Bản đồ bộ nhớ

Bảng 2.3: Bản đồ bộ nhớ

Vùng	Địa chỉ	Ghi chú
`IMEM` (ROM)	`0x0000_0000–0x0000_0FFF`	1024 lệnh, địa chỉ ô = `PC[11:2]`
`DMEM` (RAM)	`0x0000_0000–0x0000_3FFF`	16 KiB, địa chỉ ô = `Addr[13:2]`
Vùng dump của bộ test	`0x0000_0700–0x0000_077F`	`run_tests.py` lưu x0..x31 vào đây trước khi dừng
UART_TXDATA	`0xF000_0000` (ghi)	ký tự ra TTY
UART_RXDATA	`0xF000_0004` (đọc)	lấy một ký tự từ Keyboard
UART_STATUS	`0xF000_0008` (đọc)	bit 0: có ký tự chờ đọc; bit 1: TX sẵn sàng
GPIO_OUT	`0xF000_0010` (đọc/ghi)	bit [15:0] ra 4 LED 7 đoạn, bit [23:16] ra 8 LED
GPIO_IN	`0xF000_0014` (đọc)	8 công tắc DIP

Thanh ghi và PC reset về 0; phần mềm tự đặt `sp`, các chương trình test dùng `sp = 0x3FF0`.

CHƯƠNG 3. THIẾT KẾ CÁC MODULE

Mỗi module là một circuit con trong file `.circ`, có tên chân cố định và một file test vector cùng tên trong `tests/vectors/`. Mục này mô tả giao diện, cấu tạo và lý do chọn cách làm của từng module. Ảnh chụp lấy trực tiếp từ Logisim.

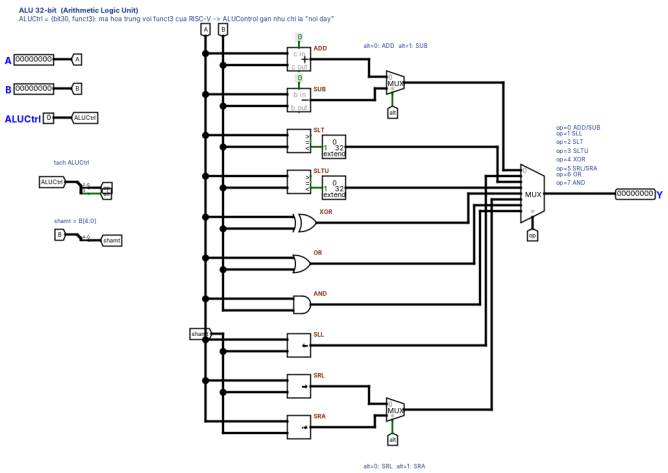
3.1 ALU và ALUControl

`ALU` nhận hai toán hạng `A`, `B` 32 bit và mã `ALUCtrl` 4 bit, cho ra `Y` 32 bit. Mười phép toán được tính song song bằng Adder, Subtractor, hai Comparator (một có dấu, một không dấu), ba cổng logic 32 bit và ba Shifter. MUX 8:1 chọn kết quả theo `ALUCtrl[2:0]`, tức funct3; hai MUX 2:1 theo `ALUCtrl[3]` chọn giữa ADD và SUB, giữa SRL và SRA. Lượng dịch chỉ lấy 5 bit thấp của `B`, đúng với đặc tả RV32I. Kết quả 1 bit của Comparator được mở rộng 0 lên 32 bit cho SLT và SLTU.

Bảng 3.1: Mã hóa `ALUCtrl`

Mã	Phép	Mã	Phép	Mã	Phép	Mã	Phép	Mã	Phép
0000	ADD	1000	SUB	0001	SLL	0010	SLT	0011	SLTU
0100	XOR	0101	SRL	1101	SRA	0110	OR	0111	AND

`ALUControl` sinh `ALUCtrl` từ `ALUOp`, funct3 và bit 30 của lệnh. Với lệnh R, bit 30 chọn SUB hoặc SRA. Với lệnh I, bit 30 thuộc immediate: lệnh `addi x1, x1, -1024` có bit 30 bằng 1. Vì vậy bit 3 của `ALUCtrl` chỉ lấy bit 30 khi lệnh là R, hoặc khi lệnh là I với funct3 = 101 (SRAI). Thiếu điều kiện này thì ADDI với immediate âm bị tính thành phép trừ; `t01_alu.s` có trường hợp kiểm tra lỗi đó.



Hình 3.1: Circuit `ALU`: mười phép toán tính song song, MUX chọn theo funct3

3.2 RegisterFile

`RegisterFile` có 31 thanh ghi 32 bit (x1 đến x31), hai cổng đọc tổ hợp `RA1`/`RD1`, `RA2`/`RD2` và một cổng ghi đồng bộ `WA`, `WD`, `WE`. Mỗi cổng đọc là một MUX 32:1 có đầu vào 0 nối hằng 0, nên x0 không phải một thanh ghi thật. Cổng ghi dùng Decoder 5→32 với enable bằng `WE`; đầu ra thứ k nối vào chân enable của thanh ghi xk, nên ở cạnh lên chỉ thanh ghi được chọn chụp giá trị. Khối còn hiển thị giá trị 32 thanh ghi ngay trên hình hộp để quan sát khi demo.

Module không tự bypass giá trị đang ghi sang cổng đọc. Trong CPU đơn chu kỳ, lệnh `addi x1, x1, 1` có rd = rs1; nếu có bypass bên trong thì đường ALU → `WD` → `RD1` → ALU thành vòng tổ hợp và mạch dao động. Pipeline cần bypass thì dùng khối `IDBypass` đặt bên ngoài (mục 5.3.1).

3.3 ImmGen

`ImmGen` tách lệnh thành 8 mảnh bit có vị trí cố định, ghép lại theo từng định dạng I, S, B, U, J, mở rộng dấu lên 32 bit rồi chọn bằng MUX theo `ImmSel` (0 = I, 1 = S, 2 = B, 3 = U, 4 = J). Do các mảnh immediate nằm cùng chỗ giữa các định dạng (imm[10:5] luôn ở bit [30:25], imm[4:1] của S và B luôn ở bit [11:8]), khối này chủ yếu là Splitter, Bit Extender và một MUX. Bit 0 của offset B và J luôn bằng 0 vì lệnh dài 4 byte.

3.4 Control

`Control` nhận lệnh 32 bit và sinh mọi tín hiệu điều khiển. Tín hiệu `valid` = `opcode[1]` AND `opcode[0]` làm enable cho Decoder 5→32 chọn theo `opcode[6:2]`. Mười đầu ra được dùng tương ứng mười nhóm lệnh; mỗi tín hiệu điều khiển là OR của các nhóm cần nó, ví dụ `RegWrite` = OP ∨ OP-IMM ∨ LOAD ∨ JAL ∨ JALR ∨ LUI ∨ AUIPC. Bảng 3.2 là bảng chân lý hiện thực trong mạch.

Bảng 3.2: Tín hiệu điều khiển theo nhóm lệnh

Nhóm	opcode	RW	ASel	BSel	ALUOp	Imm	MR	MW	WB	B/J/Jr
OP (R)	0110011	1	0	0	10	–	0	0	0	0/0/0
OP-IMM	0010011	1	0	1	11	0	0	0	0	0/0/0
LOAD	0000011	1	0	1	00	0	1	0	1	0/0/0
STORE	0100011	0	0	1	00	1	0	1	–	0/0/0
BRANCH	1100011	0	1	1	00	2	0	0	–	1/0/0
JAL	1101111	1	1	1	00	4	0	0	2	0/1/0
JALR	1100111	1	0	1	00	0	0	0	2	0/0/1
LUI	0110111	1	2	1	00	3	0	0	0	0/0/0
AUIPC	0010111	1	1	1	00	3	0	0	0	0/0/0
SYSTEM	1110011	`Halt` = 1, các tín hiệu khác bằng 0								

RW = `RegWrite`, Imm = `ImmSel`, MR = `MemRead`, MW = `MemWrite`, WB = `WBSel`, B/J/Jr = `Branch`/`Jal`/`Jalr`. ASel: 0 = rs1, 1 = PC, 2 = hằng 0. WBSel: 0 = kết quả ALU, 1 = dữ liệu bộ nhớ, 2 = PC + 4.

Ngoài các tín hiệu trên, `Control` sinh `UseRs1` và `UseRs2` cho biết lệnh có thật sự đọc rs1, rs2 hay không. Pipeline dùng hai tín hiệu này để không stall khi trường rs1 của LUI hoặc JAL tình cờ trùng rd của một lệnh load.

3.5 BranchUnit

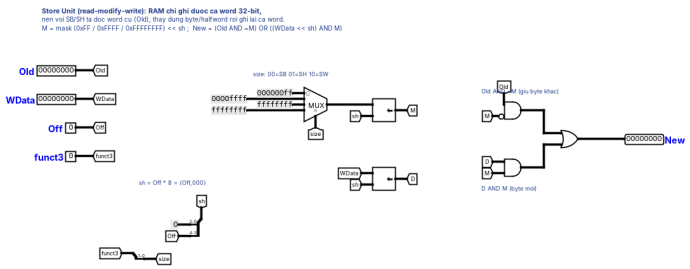
`BranchUnit` so sánh hai toán hạng bằng một Comparator không dấu (cho phép bằng và nhỏ hơn không dấu) và một Comparator có dấu. MUX 4:1 chọn theo `funct3[2:1]`: 00 là bằng, 10 là nhỏ hơn có dấu, 11 là nhỏ hơn không dấu. Kết quả XOR với `funct3[0]`, vì RISC-V mã hóa BNE, BGE, BGEU là phủ định của BEQ, BLT, BLTU, rồi AND với `Branch` để ra `Taken`.

3.6 LoadUnit, StoreUnit và AddrDecode

`AddrDecode` tách địa chỉ thành `WordAddr = Addr[13:2]`, `Off = Addr[1:0]` và `IsIO` = (`Addr[31:28] = 0xF`). `RamWE` = `MemWrite` AND NOT `IsIO`, `IoWE` = `MemWrite` AND `IsIO`.

`LoadUnit` dịch phải word đọc được `Off`$\times$8 bit để byte cần lấy về vị trí thấp nhất, mở rộng dấu hoặc mở rộng 0 theo 8 hay 16 bit, rồi chọn bằng MUX theo funct3 (000 LB, 001 LH, 010 LW, 100 LBU, 101 LHU).

`StoreUnit` làm phép đọc, sửa, ghi. Với mặt nạ M = mask $\ll$ (`Off`$\times$8), trong đó mask là `0xFF` cho SB, `0xFFFF` cho SH và `0xFFFFFFFF` cho SW, word mới là $New = (Old \wedge \neg M) \vee ((WData \ll Off \times 8) \wedge M)$. Ví dụ `sb` byte `0xAB` vào offset 2 của word `0x11223344` cho M = `0x00FF0000` và word mới `0x11AB3344`.



Hình 3.2: Circuit `StoreUnit`: ghép byte hoặc halfword mới vào word cũ

3.7 IOController

`IOController` hiện thực UART và GPIO theo kiểu memory-mapped (bảng 2.3). `Addr[4:2]` qua Decoder 3→8 (enable = `IsIO`) chọn thanh ghi I/O. Khi ghi, `TxWE` = `MemWrite` AND chọn 0 đẩy ký tự `WData[6:0]` ra TTY; thanh ghi `GPIO_OUT` 32 bit có enable = `MemWrite` AND chọn 4. Khi đọc, MUX 8:1 trả về ký tự từ Keyboard, thanh ghi trạng thái, `GPIO_OUT` hoặc 8 công tắc DIP. Ở mức CPU, MUX `IsIO` ? `IoRData` : RAM đưa dữ liệu vào `LoadUnit`, nên `lbu` hay `lh` trên thanh ghi I/O vẫn đúng.

Đọc `UART_RXDATA` lấy ký tự ra khỏi hàng đợi Keyboard. Tín hiệu `KbdRE` vì thế phải AND với `MemRead`; nếu không, một lệnh `add` tình cờ có kết quả `0xF0000004` cũng làm mất ký tự. Trong pipeline, việc đọc chỉ xảy ra ở tầng MEM, nơi lệnh chắc chắn đã qua điểm flush, nên thanh ghi EX/MEM phải mang theo trường `MemRead`. UART dùng hỏi vòng (polling) thanh ghi trạng thái, không có ngắt.

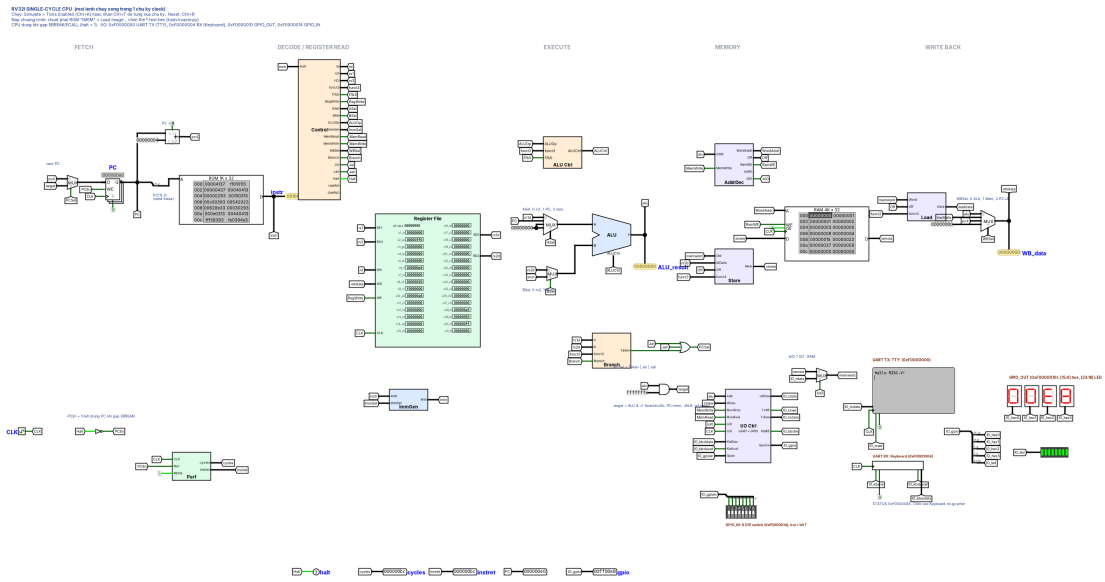
3.8 PerfCounters

`PerfCounters` có hai bộ đếm 32 bit. `cycles` tăng ở mỗi cạnh lên khi CPU đang chạy (`Run` = 1, tức chưa halt). `instret` tăng khi có lệnh hoàn thành (`Retire` = 1): ở SC là mỗi chu kỳ, ở PL là khi lệnh ở tầng WB có `valid` = 1, nên bubble không được đếm. Hai giá trị được đưa ra chân `cycles`, `instret` để bộ test đọc và tính CPI.

CHƯƠNG 4. CPU ĐƠN CHU KỲ

4.1 Datapath

Trong CPU đơn chu kỳ, toàn bộ đường từ thanh ghi `PC` qua các module đến register file và RAM là mạch tổ hợp. Mỗi lệnh xong trong một chu kỳ: ở cạnh lên của clock, `PC`, thanh ghi đích và RAM cùng cập nhật. Hình 4.1 là mạch top-level `RV32I_SingleCycle`.



Hình 4.1: Mạch top-level của CPU đơn chu kỳ trong Logisim

Một lệnh đi qua năm bước, tất cả trong cùng một chu kỳ:

1. Nạp lệnh: `PC` đưa vào `IMEM` (`PC[11:2]`) cho ra `instr`; song song đó bộ cộng tính PC + 4.
2. Giải mã: `Control` sinh tín hiệu điều khiển, `RegisterFile` đọc rs1 và rs2, `ImmGen` tạo immediate.
3. Thực thi: A = rs1, PC hoặc 0 theo `ASel`; B = rs2 hoặc immediate theo `BSel`; `ALU` tính, `BranchUnit` so sánh rs1 với rs2.
4. Truy cập bộ nhớ: địa chỉ là kết quả ALU. `StoreUnit` ghép dữ liệu ghi, RAM ghi ở cạnh lên nếu `RamWE` = 1, `LoadUnit` tách dữ liệu đọc.
5. Ghi lại: `WBSel` chọn kết quả ALU, dữ liệu load hoặc PC + 4; register file ghi vào rd ở cạnh lên.

PC kế tiếp được chọn bởi `PCSel` = `Taken` ∨ `Jal` ∨ `Jalr`: bằng 1 thì PC nhận kết quả ALU AND `0xFFFFFFFE`, bằng 0 thì nhận PC + 4. Đặc tả JALR yêu cầu xóa bit 0 của đích nhảy [1]; với branch và JAL bit này vốn bằng 0, nên một

mạch dùng chung cho cả ba. Thanh ghi `PC` có enable = NOT `Halt`, nên khi gặp EBREAK, CPU giữ nguyên trạng thái và chân `halt` lên 1.

4.2 Ví dụ: lệnh `lw t4, 8(sp)`

Lệnh này có mã máy `0x00812E83`: immediate = 8, rs1 = x2, funct3 = 010, rd = x29, opcode = 0000011 (LOAD). `Control` bật nhóm LOAD và cho `RegWrite` = 1, `ASel` = 0, `BSel` = 1, `ImmSel` = 0, `MemRead` = 1, `MemWrite` = 0, `WBSel` = 1. ALU cộng sp với 8 ra địa chỉ, `AddrDecode` thấy `IsIO` = 0 nên đọc RAM, `LoadUnit` chọn cả word theo funct3 = 010, MUX ghi lại chọn dữ liệu bộ nhớ, và ở cạnh lên t4 nhận giá trị. `PCSel` = 0 nên PC nhận PC + 4.

4.3 Đường găng và giới hạn của thiết kế đơn chu kỳ

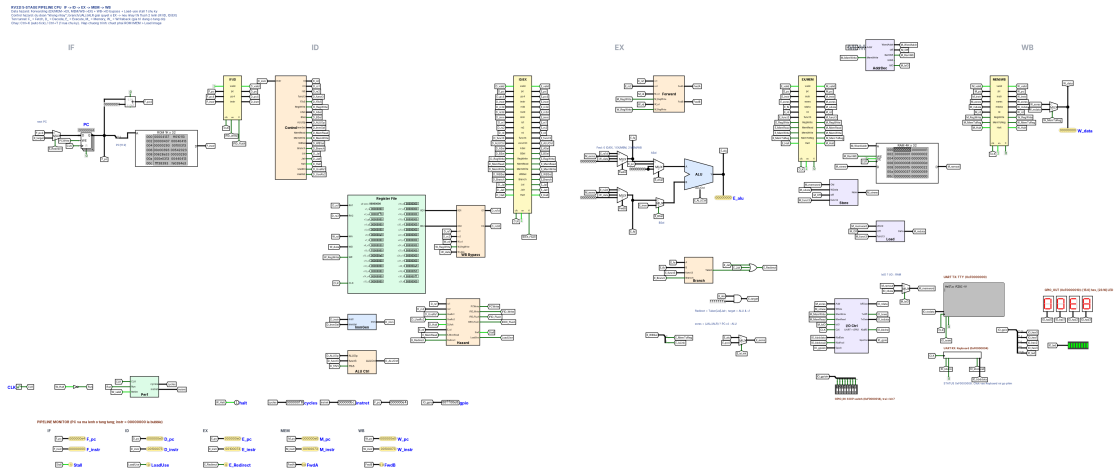
Chu kỳ clock phải đủ dài cho đường tổ hợp dài nhất. Lệnh load đi qua nhiều khối nhất: `IMEM`, `Control` và `RegisterFile`, `ALU`, `DMEM`, `LoadUnit`, MUX ghi lại, rồi đến đầu vào register file. Các lệnh ngắn hơn như `add` không dùng RAM nhưng vẫn phải chờ hết chu kỳ đó.

Logisim không mô phỏng trễ cổng, nên báo cáo dùng giả định trễ từng bước theo Patterson và Hennessy [3, ch. 4]: nạp lệnh 200 ps, đọc thanh ghi 100 ps, ALU 200 ps, truy cập bộ nhớ dữ liệu 200 ps, ghi thanh ghi 100 ps. Với giả định này, chu kỳ đơn chu kỳ là 800 ps. Đây là con số giả định để so sánh, không phải số đo trên mạch. Chương 5 chia đường này thành năm tầng để rút ngắn chu kỳ clock.

CHƯƠNG 5. CPU PIPELINE 5 TẦNG VÀ XỬ LÝ HAZARD

5.1 Chia datapath thành năm tầng

Pipeline cắt đường tổ hợp của CPU đơn chu kỳ thành năm đoạn IF, ID, EX, MEM, WB và đặt một thanh ghi pipeline giữa hai đoạn liên tiếp. Mỗi chu kỳ, năm lệnh khác nhau nằm ở năm tầng. Chu kỳ clock chỉ cần đủ cho tầng chậm nhất cộng với thời gian của thanh ghi pipeline. Với giả định trễ ở mục 4.3 và 20 ps cho thanh ghi pipeline, chu kỳ pipeline là $\max(200, 100, 200, 200, 100) + 20 = 220$ ps, so với 800 ps của bản đơn chu kỳ. Nếu mọi lệnh đều có CPI bằng 1, tăng tốc lý tưởng là $800/220 \approx 3.64$ lần.



Hình 5.1: Mạch top-level của CPU pipeline. Pipeline Monitor ở góc dưới hiển thị PC và lệnh ở từng tầng cùng các tín hiệu `Stall`, `LoadUse`, `E_Redirect`, `FwdA`, `FwdB`

5.2 Thanh ghi pipeline

Mỗi thanh ghi pipeline giữ mọi thứ các tầng sau cần về một lệnh: dữ liệu, số hiệu thanh ghi và các tín hiệu điều khiển. Tín hiệu điều khiển được sinh một lần ở ID rồi đi theo lệnh qua từng tầng. Mỗi trường gồm một MUX `FLUSH` ? 0 : D và một Register có enable = `EN` OR `FLUSH`. Khi `EN` = 0 thanh ghi giữ giá trị (stall). Khi `FLUSH` = 1 thanh ghi nạp toàn 0; do lệnh 0 có mọi tín hiệu điều khiển bằng 0, lệnh đó thành bubble và không ghi gì. Flush được làm đồng bộ qua MUX vì chân Clear của Register trong Logisim là bất đồng bộ.

Bảng 5.1: Các trường của thanh ghi pipeline

Thanh ghi	Trường
IF/ID	valid, pc, pc4, instr
ID/EX	valid, pc, pc4, instr, rs1d, rs2d, imm, rs1, rs2, rd, funct3, ALUCtrl, ASel, BSel, RegWrite, MemRead, MemWrite, WBSel, Branch, Jal, Jalr, Halt
EX/MEM	valid, pc, instr, exres, sdata, rd, funct3, RegWrite, MemRead, MemWrite, MemToReg, Halt
MEM/WB	valid, pc, instr, exres, mdata, rd, RegWrite, MemToReg, Halt

Trường `exres` là PC + 4 với JAL/JALR và kết quả ALU với các lệnh khác. Nó vừa là địa chỉ bộ nhớ, vừa là giá trị để forward, vừa là giá trị ghi lại, nên từ tầng MEM trở đi chỉ cần MUX 2:1 theo `MemToReg`. Trường `valid` bằng 0 với bubble và dùng cho bộ đếm `instret`. Các trường `pc`, `instr` ở mọi tầng chỉ để hiển thị trên Pipeline Monitor.

5.3 Ba loại hazard

Structural hazard không xảy ra vì bộ nhớ lệnh và bộ nhớ dữ liệu tách riêng, register file có hai cổng đọc và một cổng ghi. Data hazard xảy ra khi một lệnh cần kết quả của lệnh trước mà lệnh đó chưa ghi vào register file. Control hazard xảy ra vì pipeline đã nạp tiếp hai lệnh sau một lệnh nhảy trước khi biết lệnh đó có nhảy hay không.

5.3.1 Forwarding và bypass WB→ID

Kết quả của một lệnh ALU đã có ở cuối tầng EX, chỉ chưa được ghi vào register file. `ForwardUnit` so sánh rs1, rs2 của lệnh ở EX với rd của lệnh ở MEM và WB. `FwdA` = 01 (lấy từ EX/MEM) khi EX/MEM.RegWrite = 1, EX/MEM.rd $\neq 0$ và EX/MEM.rd = ID/EX.rs1. `FwdA` = 10 (lấy từ MEM/WB) khi điều kiện tương tự với MEM/WB đúng còn điều kiện 01 sai. Các trường hợp khác cho `FwdA` = 00, dùng giá trị đọc ở ID. `FwdB` làm tương tự với rs2.

Điều kiện rd $\neq 0$ ngăn lệnh `add x0, ...` forward một giá trị khác 0 cho lệnh sau đọc x0. EX/MEM được ưu tiên vì khi hai lệnh liên tiếp cùng ghi một thanh ghi, giá trị mới nhất thuộc lệnh gần hơn. MUX forwarding đặt trước MUX `ASel`/`BSel`, nên giá trị đã forward dùng được cho ALU, cho `BranchUnit` và làm dữ liệu store.

Còn trường hợp lệnh cách 3 vị trí: lệnh ở WB ghi register file ở cạnh lên cuối chu kỳ, trong khi lệnh ở ID đọc ngay trong chu kỳ đó và nhận giá trị cũ. `IDBypass` đặt sau cổng đọc của register file: nếu W_RegWrite = 1, W_rd $\neq 0$ và W_rd trùng rs thì đưa thẳng W_data vào ID. Cách này tương đương với quy ước ghi nửa đầu,

đọc nửa sau chu kỳ trong sách [3, ch. 4], và giữ `RegisterFile` dùng được cho CPU đơn chu kỳ.

5.3.2 Load-use: stall một chu kỳ

Dữ liệu của lệnh load chỉ có ở cuối tầng MEM, còn lệnh ngay sau cần nó ở đầu tầng EX trong cùng chu kỳ, nên forwarding không đủ. `HazardUnit` phát hiện trường hợp này:

$$\begin{aligned} LoadUse = {} & E_MemRead \wedge (E_rd \neq 0) \\ & \wedge \big((E_rd = rs1 \wedge UseRs1) \vee (E_rd = rs2 \wedge UseRs2)\big) \end{aligned}$$

Khi `LoadUse` = 1, `PCWrite` = 0 và `IFID_Write` = 0 giữ nguyên PC và IF/ID, còn `IDEX_Flush` = 1 chèn một bubble vào EX. Chu kỳ sau, lệnh phụ thuộc lên EX và nhận dữ liệu load qua forwarding từ MEM/WB.

5.3.3 Control hazard: dự đoán không nhảy và flush

PL luôn nạp PC + 4. Lệnh rẽ nhánh và nhảy được quyết định ở EX: `E_Redirect` = `Taken` ∨ `Jal` ∨ `Jalr`. Khi `E_Redirect` = 1, PC nhận địa chỉ đích và hai lệnh nạp sai ở IF/ID và ID/EX bị flush, mất 2 chu kỳ. Nếu cùng chu kỳ có cả stall và redirect thì redirect thắng, vì lệnh đang bị stall nằm trên nhánh sai. Các phương trình của `HazardUnit` là:

$$Stall = LoadUse \vee D_Halt, \quad PCWrite = IFID_Write = \neg Stall \vee Redirect,$$

$$IFID_Flush = Redirect, \quad IDEX_Flush = Redirect \vee LoadUse.$$

EBREAK ở ID làm `Stall` = 1 để ngừng nạp lệnh mới. Chân `halt` lấy từ trường `Halt` của MEM/WB, nên CPU chỉ dừng khi EBREAK tới WB, lúc mọi lệnh trước đó đã ghi xong.

Từ các cơ chế trên, một chương trình N lệnh chạy trên PL mất $N + 4 +$ (số lần load-use) $+ 2 \times$ (số lần nhảy) chu kỳ, trong đó 4 là số chu kỳ lấp đầy pipeline. Bộ test so đúng con số này với bộ đếm `cycles` của Logisim trên mọi chương trình (chương 6).

5.4 Bản NoHazard và cách chèn NOP

Để đo lợi ích của phần cứng xử lý hazard, nhóm giữ một bản pipeline có hai thay đổi so với PL: chân chọn của MUX forwarding nối hằng 0, và đầu vào

`E_MemRead` của `HazardUnit` nối hằng 0. Flush khi nhảy và `IDBypass` vẫn giữ. Chạy chương trình bình thường trên bản này thì sai: cả 12 chương trình test đều FAIL (mục 6.4).

Để bản NH chạy đúng, assembler có chế độ `-nohazard` chèn `nop` theo hai quy tắc. Lệnh đọc rd của lệnh ngay trước được chèn 2 `nop`; lệnh đọc rd của lệnh trước đó 2 vị trí được chèn 1 `nop`. Sau đó khoảng cách giữa hai lệnh là 3 và `IDBypass` lo phần còn lại. Bộ đếm khoảng cách được đặt lại sau `jal` và `jalr`, vì lệnh đích chỉ đến được qua một lần nhảy và mỗi lần nhảy đã flush 2 chu kỳ.

Bảng 5.2 so sánh cùng bảy lệnh của `t06_hazard.s` trên hai bản. Ô màu đỏ là chu kỳ lệnh bị stall ở ID; bảng sinh tự động từ golden model.

Bảng 5.2: Giản đồ pipeline của 7 lệnh trong `t06_hazard.s`

(a) PL: forwarding và stall bằng phần cứng

Lệnh	1	2	3	4	5	6	7	8	9	10	11	12	13	14	Ghi chú
`sw t3, 0(sp)`	IF	ID	EX	ME	WB										
`lw t4, 0(sp)`		IF	ID	EX	ME	WB									
`addi t5, t4, 1`			IF	ID	ID	EX	ME	WB							stall
`lw t6, 0(sp)`					IF	ID	EX	ME	WB						
`sw t6, 4(sp)`						IF	ID	ID	EX	ME	WB				stall
`lw a0, 4(sp)`								IF	ID	EX	ME	WB			
`beq a0, t3, ok1`									IF	ID	ID	EX	ME	WB	stall, nhảy
`lui s11, 1`											IF	ID			bị flush
`addi s11, s11, -1107`												IF			bị flush

(b) NH: không forwarding, assembler chèn `nop`

Lệnh	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	Ghi chú
`sw t3, 0(sp)`	IF	ID	EX	ME	WB													
`lw t4, 0(sp)`		IF	ID	EX	ME	WB												
nop (chèn)			IF	ID	EX	ME	WB											
nop (chèn)				IF	ID	EX	ME	WB										
`addi t5, t4, 1`					IF	ID	EX	ME	WB									
`lw t6, 0(sp)`						IF	ID	EX	ME	WB								
nop (chèn)							IF	ID	EX	ME	WB							
nop (chèn)								IF	ID	EX	ME	WB						
`sw t6, 4(sp)`									IF	ID	EX	ME	WB					
`lw a0, 4(sp)`										IF	ID	EX	ME	WB				
nop (chèn)											IF	ID	EX	ME	WB			
nop (chèn)												IF	ID	EX	ME	WB		
`beq a0, t3, ok1`													IF	ID	EX	ME	WB	nhảy
`lui s11, 1`														IF	ID			bị flush
`nop`															IF			bị flush

Trên PL, ba lệnh dùng ngay kết quả của `lw` mỗi lệnh chờ 1 chu kỳ, các phụ thuộc khác được forward. Lệnh `beq` nhảy ở EX nên hai lệnh phía sau bị flush. Bảy lệnh chiếm 14 chu kỳ từ lúc `sw` vào IF đến lúc `beq` rời WB. Trên NH, mỗi phụ thuộc gần phải chờ bằng `nop`, kể cả phụ thuộc mà PL forward được không mất chu kỳ nào, nên cùng đoạn này chiếm 17 chu kỳ.

CHƯƠNG 6. KIỂM THỬ

Kiểm thử chia hai mức: test vector cho từng module và chương trình chạy trên toàn CPU. Mọi kết quả trong chương này được chạy lại ngày 05/11/2026 trên Logisim-evolution 4.1.0, Java 21, Windows 11.

6.1 Toolchain

Assembler `tools/rvasm.py` dịch toàn bộ RV32I, các lệnh giả thông dụng (`li`, `la`, `call`, `ret`, ...) và các directive `.text`, `.data`, `.word`, `.asciz`, ... Đầu ra là `.text.hex` và `.data.hex` để nạp ROM, RAM bằng *Load Image*, kèm file `.lst`. Tùy chọn `-circ` ghi thẳng mã máy vào ROM `IMEM` trong file `.circ`; `-nohazard` chèn `nop` cho bản NH (mục 5.4). Lệnh giả `li` với hằng 32 bit tách thành `lui` và `addi`; vì `addi` mở rộng dấu, khi bit 11 của phần thấp bằng 1 thì phần cao phải cộng thêm 1, và `t02_upper.s` kiểm tra trường hợp hằng `0x800`.

Golden model `tools/rvsim.py` là trình mô phỏng mức lệnh viết bằng Python, không dùng gì từ file mạch. Nó ghi lại 32 thanh ghi, toàn bộ RAM, chuỗi TTY, GPIO_OUT, và đếm số lần load-use, số lần nhảy để tính số chu kỳ của PL theo công thức ở chương 5. Với SC, số chu kỳ bằng số lệnh.

Bộ chạy test `tools/run_tests.py` thay EBREAK cuối chương trình bằng đoạn mã lưu x1 đến x31 vào vùng `0x704–0x77C`, ghi mã máy vào ROM của một bản sao file `.circ`, chạy Logisim không giao diện rồi so với golden model: 31 thanh ghi, 16 KiB RAM, chuỗi TTY, GPIO_OUT và hai bộ đếm `cycles`, `instret`. Chương trình có dòng `# STDIN:` được gõ chuỗi đó vào Keyboard và không so số chu kỳ, vì thời điểm ký tự đến phụ thuộc Logisim. Vì so cả số chu kỳ, bộ test bắt được lỗi thời gian mà kết quả cuối vẫn đúng, ví dụ stall thừa hoặc flush thiếu.

6.2 Test vector từng module

Mỗi file trong `tests/vectors/` có dòng tiêu đề là tên chân và mỗi dòng sau là một bộ giá trị vào cùng giá trị ra mong đợi. Giá trị mong đợi được sinh từ mô hình Python trong `generator/refmodels.py`. Hai module tuần tự `RegisterFile` và `IOController` có cột `CLK` trong vector để tạo cạnh lên. Bảng 6.1 là kết quả chạy bằng tùy chọn `-w` của Logisim (phụ lục A).

Bảng 6.1: Kết quả test vector (đạt / tổng số); ba khối cuối chỉ có ở PL

Module	Nội dung vector	SC	PL
`ALU`	ngẫu nhiên và giá trị biên	250/250	250/250
`ALUControl`	vét cạn 64 tổ hợp	64/64	64/64
`ImmGen`	lệnh ngẫu nhiên với mọi `ImmSel`	300/300	300/300
`Control`	gồm lệnh không hợp lệ và lệnh 0	400/400	400/400
`RegisterFile`	tuần tự, 200 bước ghi và đọc	400/400	400/400
`BranchUnit`	20% cặp toán hạng bằng nhau	300/300	300/300
`LoadUnit`	funct3 và offset 0–3 ngẫu nhiên	300/300	300/300
`StoreUnit`	funct3 và offset 0–3 ngẫu nhiên	300/300	300/300
`AddrDecode`	địa chỉ RAM và I/O	200/200	200/200
`IOController`	tuần tự, 200 bước đọc ghi	400/400	400/400
`ForwardUnit`	số hiệu thanh ghi dồn vào x0, x1, x2, x5	–	400/400
`HazardUnit`	như trên, thêm `Redirect`, `D_Halt`	–	400/400
`IDBypass`	như trên, dữ liệu ngẫu nhiên	–	300/300
Tổng		2914/2914	4014/4014

6.3 Chương trình test toàn CPU

Thư mục `programs/` có 12 chương trình. `demo.s` nạp sẵn trong ROM, in `Hello RISC-V!` và hiện tổng 12 số Fibonacci lên LED 7 đoạn. `t01_alu.s` đến `t05_mem.s` phủ từng nhóm lệnh: lệnh R và I số học (gồm ADDI với immediate âm), LUI và AUIPC, sáu lệnh rẽ nhánh, JAL và JALR, load và store byte, halfword, word. `t06_hazard.s` gom các trường hợp forwarding khoảng cách 1, 2, 3, load-use, ghi x0 và JALR dùng thanh ghi vừa load. `t07_fib.s`, `t08_sort.s` (sắp xếp nổi bọt) và `t09_recursion.s` (đệ quy dùng stack) là chương trình thực tế hơn. `t10_hello.s` và `t11_io.s` kiểm tra UART và GPIO.

Ngoài ra `tools/fuzz.py` sinh 25 chương trình ngẫu nhiên (seed 0 đến 24, mỗi chương trình 189 đến 231 lệnh) với nhiều phụ thuộc gần nhau để kích hoạt forwarding và stall.

6.4 Kết quả

Bảng 6.2: Kết quả chạy `run_tests.py`

File mạch	Chế độ assemble	12 chương trình	25 chương trình fuzz
SC	bình thường	12/12 PASS	25/25 PASS
PL	bình thường	12/12 PASS	25/25 PASS
NH	bình thường	0/12 PASS	không chạy
NH	`-nohazard`	12/12 PASS	25/25 PASS

Ở mọi lần PASS, số chu kỳ Logisim đếm khớp với golden model (trừ `t11_io.s`, không so số chu kỳ). Bản NH chạy chương trình bình thường thì sai ngay từ `demo.s`: thanh ghi `sp` ra `0xFFFFFFF0` thay vì `0x00003FF0`, vì `addi sp, sp, -16` đọc `sp` trước khi `lui sp, 4` kịp ghi.

CHƯƠNG 7. KẾT QUẢ VÀ ĐÁNH GIÁ

7.1 Số chu kỳ của ba phiên bản

Bảng 7.1 là số chu kỳ của 12 chương trình trên ba phiên bản. Số liệu lấy từ golden model, không tính đoạn dump 32 thanh ghi mà `run_tests.py` chèn thêm. Mục 6.4 đã kiểm tra mô hình này khớp từng chu kỳ với Logisim trên cả ba file mạch. Cột "NOP" là số lệnh `nop` do assembler chèn mà bản NH thực sự thực thi; cột "Load-use" và "Nhảy" là số lần stall và số lần rẽ nhánh hoặc nhảy được thực hiện trên PL.

Bảng 7.1: Số lệnh và số chu kỳ trên SC, NH và PL

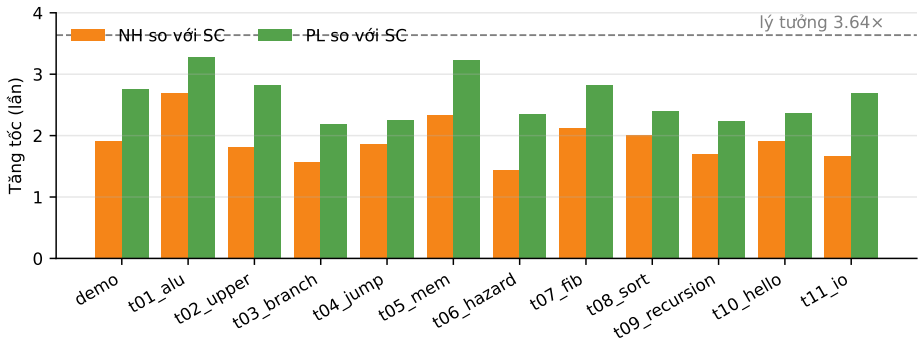
Chương trình	Lệnh	SC	NH	NOP	PL	Load-use	Nhảy	CPI (PL)
`demo.s`	188	188	358	122	248	12	22	1.32
`t01_alu.s`	37	37	50	9	41	0	0	1.11
`t02_upper.s`	14	14	28	10	18	0	0	1.29
`t03_branch.s`	54	54	125	35	90	0	16	1.67
`t04_jump.s`	84	84	164	28	136	0	24	1.62
`t05_mem.s`	32	32	50	14	36	0	0	1.12
`t06_hazard.s`	49	49	124	53	76	5	9	1.55
`t07_fib.s`	146	146	250	62	188	0	19	1.29
`t08_sort.s`	424	424	766	168	643	45	85	1.52
`t09_recursion.s`	4765	4765	10184	2895	7753	464	1260	1.63
`t10_hello.s`	97	97	185	52	149	16	16	1.54
`t11_io.s`	103	103	225	94	139	8	12	1.35
Tổng	5993	5993	12509	3542	9517	550	1463	1.59

Trên SC, CPI luôn bằng 1. Trên PL, CPI của cả 12 chương trình là 1.59, cao nhất ở `t09_recursion.s` với 1.63. Bản NH có CPI tính trên số lệnh gốc là 2.09: tổng cộng nó thực thi 3542 lệnh `nop`, trong khi PL chỉ stall 550 chu kỳ cho load-use.

Số chu kỳ thêm của PL so với số lệnh đến từ hai nguồn có thể đếm chính xác. Mỗi lần nhảy mất 2 chu kỳ flush, mỗi lần load-use mất 1 chu kỳ stall, cộng 4 chu kỳ lấp đầy pipeline. Trên `t09_recursion.s`, 1260 lần nhảy tốn 2520 chu kỳ, gấp hơn 5 lần 464 chu kỳ của load-use. Chương trình đệ quy có nhiều lệnh gọi, trả về và rẽ nhánh, nên control hazard là nguồn mất mát lớn nhất. Ở NH, mọi phụ thuộc gần đều phải chờ bằng `nop`, kể cả phụ thuộc ALU sang ALU mà PL forward được không mất chu kỳ nào.

7.2 Thời gian thực thi theo giả định trễ

Số chu kỳ chưa đủ để so sánh, vì chu kỳ clock của ba bản khác nhau. Logisim không mô phỏng trễ cổng, nên hình 7.1 dùng giả định trễ đã nêu ở mục 4.3: 800 ps cho SC và 220 ps cho NH và PL. Thời gian bằng số chu kỳ nhân chu kỳ clock. Các con số này là ước tính theo giả định, không phải số đo.



Hình 7.1: Tăng tốc so với SC theo giả định trễ; đường đứt nét là mức lý tưởng khi CPI = 1

Tính trên tổng 12 chương trình, PL nhanh hơn SC 2.29 lần và NH nhanh hơn SC 1.74 lần, so với mức lý tưởng 3.64 lần. PL thực thi ít chu kỳ hơn NH 1.31 lần với cùng chu kỳ clock; đây là phần đóng góp riêng của `ForwardUnit` và load-use stall bằng phần cứng.

Khoảng cách giữa 2.29 và 3.64 có hai nguyên nhân. Thứ nhất, CPI của PL lớn hơn 1 vì flush và stall như đã phân tích. Thứ hai, năm tầng không dài bằng nhau: tầng ID và WB chỉ cần 100 ps nhưng vẫn chiếm trọn 220 ps. Giả thuyết cần kiểm chứng là đưa quyết định rẽ nhánh lên ID hoặc thêm dự đoán nhánh sẽ thu hẹp khoảng cách này nhiều nhất trên các chương trình có nhiều lần nhảy như `t09_recursion.s`; mục 8.3 ước lượng phần lợi đó.

7.3 Một lần chạy cụ thể

Chương trình `demo.s` nạp sẵn trong ROM của cả ba file mạch có 188 lệnh. Trên SC nó chạy 188 chu kỳ. Trên PL nó chạy 248 chu kỳ, gồm 12 chu kỳ stall và 22 lần nhảy. Trên NH, sau khi assembler chèn `nop`, nó chạy 358 chu kỳ, trong đó 122 chu kỳ là `nop`. Cả ba bản kết thúc với TTY in `Hello RISC-V!`, thanh ghi `a0` = `0xE8`, LED 7 đoạn hiện `00E8` và 8 LED sáng.

CHƯƠNG 8. PHẦN MỞ RỘNG, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

8.1 Phần mở rộng đã làm

Ngoài yêu cầu cơ bản, nhóm làm thêm các lệnh tùy chọn, UART và GPIO, forwarding hai nguồn cùng bypass WB→ID, bộ đếm `cycles`, `instret`, Pipeline Monitor, toolchain kiểm thử tự động và bản NH (bảng 1.1).

8.2 Hạn chế

- Truy cập bộ nhớ lệch (ví dụ `lw` ở địa chỉ không chia hết cho 4) không được hỗ trợ, kết quả không xác định.
- Không có exception, ngắt và CSR. ECALL, EBREAK được dùng làm lệnh dừng; FENCE thực thi như NOP. UART vì thế phải hỏi vòng thanh ghi trạng thái.
- Vùng I/O chỉ giải mã `Addr[31:28]` và `Addr[4:2]`, nên mọi địa chỉ `0xFxxx_xxxx` khác là bí danh của 8 thanh ghi I/O.
- Rẽ nhánh quyết định ở EX nên mỗi lần nhảy mất 2 chu kỳ. Trên 12 chương trình test, flush chiếm $2 \times 1463 = 2926$ chu kỳ, nhiều hơn tổng 550 chu kỳ của load-use.

8.3 Hướng phát triển

Bảng 8.1 xếp các mục nâng cao chưa làm theo lợi ích đo được bằng bộ test hiện có. Cache không có trong bảng vì RAM của Logisim trả dữ liệu ngay trong chu kỳ, nên không đo được lợi ích.

Bảng 8.1: Hướng phát triển và cách đo lợi ích

Mở rộng	Cách làm	Lợi ích có thể đo
Quyết định rẽ nhánh ở ID	Đưa `BranchUnit` và bộ cộng địa chỉ đích lên ID, thêm forwarding vào ID	Phạt mỗi lần nhảy giảm từ 2 xuống 1 chu kỳ. Trên `t09_recursion.s` tiết kiệm tối đa 1260 chu kỳ; phải trừ đi các stall mới khi lệnh rẽ nhánh phụ thuộc lệnh ngay trước
Dự đoán nhánh 2 bit và BTB	Bảng nhỏ đánh chỉ số bằng `PC[5:2]`, lưu địa chỉ đích và bộ đếm bão hòa 2 bit	Giảm flush ở vòng lặp (`t08_sort.s`, `t09_recursion.s`); đo bằng số lần `E_Redirect`
Phần mở rộng M (MUL, DIV)	Thêm Multiplier, Divider của Logisim vào ALU, chọn theo funct7 = 0000001	`t09_recursion.s` đang nhân bằng vòng lặp dịch và cộng; thay bằng một lệnh `mul`
Ngắt và CSR	Thêm `mtvec`, `mepc`, `mcause` và các lệnh CSR	UART nhận ký tự không cần hỏi vòng

TÀI LIỆU THAM KHẢO

[1] A. Waterman and K. Asanović, Eds., *The RISC-V instruction set manual, volume I: Unprivileged ISA, document version 20191213*, RISC-V Foundation, 2019. Accessed: Nov. 5, 2026. [Online]. Available: `https://github.com/riscv/riscv-isa-manual`

[2] Logisim-evolution developers, *Logisim-evolution*, version 4.1.0. Accessed: Nov. 5, 2026. [Online]. Available: `https://github.com/logisim-evolution/logisim-evolution`

[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2017.

PHỤ LỤC

A. HƯỚNG DẪN SỬ DỤNG PROJECT

Phụ lục này tóm tắt `README.md`. Yêu cầu phần mềm: Logisim-evolution 4.1.0 (cần Java 21) và Python 3.8 trở lên. Assembler và bộ test không cần thư viện Python ngoài; riêng script sinh biểu đồ của báo cáo cần `matplotlib`.

A.1 Mở project và chạy mô phỏng

1. Mở Logisim-evolution 4.1.0, chọn File → Open rồi chọn một trong hai file:
   `circ/RV32I_SingleCycle.circ`
   `circ/RV32I_Pipeline.circ`
2. Ctrl+K cho clock tự chạy, chỉnh tốc độ ở Simulate → Tick Frequency. Ctrl+T chạy từng nửa chu kỳ, Ctrl+R reset.
3. Chương trình `demo.s` đã nạp sẵn trong ROM. Khi chạy xong, TTY in `Hello RISC-V!`, LED 7 đoạn hiện `00E8`, 8 LED sáng và chân `halt` lên 1.
4. Quan sát: 32 thanh ghi hiện trên khối `RegisterFile`; RAM xem trên khối `DMEM`; ở bản pipeline, Pipeline Monitor hiện PC và lệnh ở từng tầng cùng `Stall`, `LoadUse`, `E_Redirect`, `FwdA`, `FwdB`.

A.2 Nạp chương trình mới

```
python tools/rvasm.py programs/t08_sort.s -o build/t08_sort
```

Lệnh trên sinh `build/t08_sort.text.hex`, `.data.hex` và `.lst`. Trong Logisim, chuột phải ROM `IMEM` → Load Image và chọn file `.text.hex`; nếu chương trình có `.data` thì làm tương tự với RAM `DMEM` và file `.data.hex`. Có thể ghi thẳng vào file mạch:

```
python tools/rvasm.py programs/t08_sort.s -o build/t08_sort \
    --circ circ/RV32I_Pipeline.circ --circ-out
   build/sort_pipeline.circ
```

Với bản NH, thêm `-nohazard` để assembler chèn `nop`.

A.3 Chạy test

```
export LOGISIM_JAR=/path/to/logisim-evolution-4.1.0-all.jar
python tools/run_tests.py circ/RV32I_SingleCycle.circ
python tools/run_tests.py circ/RV32I_Pipeline.circ
python tools/run_tests.py circ/RV32I_Pipeline.circ --programs
   programs/fuzz
python tools/run_tests.py
   circ/extra/RV32I_Pipeline_NoHazard.circ --nohazard
java -jar $LOGISIM_JAR -w ALU tests/vectors/ALU.txt
   circ/RV32I_Pipeline.circ
```

```
python tools/compare.py programs/t06_hazard.s
```

Lệnh `-w` chạy test vector của một module và in `Passed: n, Failed: m`. Lệnh này không chạy được khi thư mục chứa file mạch có file autosave của một phiên Logisim đang mở (`.RV32I_Pipeline.circ.autosave`); khi đó Logisim cần hỏi có khôi phục bản autosave không. Đóng Logisim hoặc chạy trên một bản sao của file mạch.

A.4 Dựng lại báo cáo

```
python latex/scripts/gen_results.py
cd latex && latexmk -pdf report.tex
```

Script thứ nhất sinh lại các bảng số chu kỳ, giản đồ pipeline và biểu đồ trong `latex/generated/` từ golden model. Lệnh thứ hai biên dịch báo cáo (pdfLaTeX và biber).

A.5 Kịch bản demo

1. Mở `RV32I_SingleCycle.circ`, giới thiệu sơ đồ khối. Chạy `demo.s`, chỉ vào `RegisterFile`, TTY và LED 7 đoạn. Nạp `t11_io.s`, click vào khối Keyboard, gõ chữ thường và xem TTY in lại chữ hoa.
2. Mở `RV32I_Pipeline.circ`, chạy từng chu kỳ tới một lần load-use: Pipeline Monitor cho `Stall` = 1 và bubble `00000000` ở EX. Chạy tới một lần rẽ nhánh nhảy: `E_Redirect` = 1 và hai lệnh phía sau thành `00000000`.
3. Chạy `run_tests.py` trên terminal, cho thấy `ALL PASSED` với SC và PL.
4. Chạy `run_tests.py` trên bản NH không có `-nohazard` để thấy FAIL, rồi chạy lại với `-nohazard` để thấy PASS.